

- Each player should draw with a different colour to identify the winner.
- The name of the player who has made the last move is displayed.
- Upon winning, the winner is notified.
- The game board should be cleared using the *Wipe* button.

So let's move on and add these behaviours using the block editor. I hope you remember how to switch from designer to block editor. There is a button available right above the *Properties* pane to switch between the designer and block editor.

Block editor blocks

I have already prepared the blocks for you. All you have to do is drag the relevant blocks from the left side palette and drop them on to the viewer. Arrange the blocks in the same way you see them in Figure 5. I will explain each one — what it does and how it is called. Initialise three variables as follows:

- KEY_LINE:** This will be used for storing data points about the lines, which will be further used to draw coloured lines.
- KEY_USER:** This variable will be used to keep track of the last user, so it will be storing each username at every turn.
- KEY_COMMAND:** This variable will be used to determine the colour of the pen the user has picked for drawing.

Now, before the user draws anything on the canvas or chooses any colour for the pen, we should check whether he/she has provided a name. If there is a name in the textbox at the top, we will save it to the Firebase cloud database. The procedure (Figure 6) checks for the name and, if present, saves it to the database.

We want each user to draw with a different coloured pen so that it will be easy to identify the moves. So we have included buttons to set the colour of the pen. Now we want to display this colour setting on each device; hence, it should be stored in Firebase. Figure 7 shows the code for one button, and I hope you can make it for the other two as well.



Figure 5: Block editor image 1



Figure 6: Block editor image 2



Figure 7: Block editor image 3



Figure 8: Block editor image 4

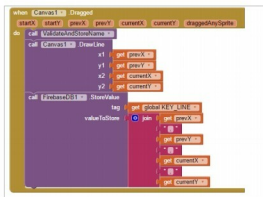


Figure 9: Block editor image 5

Once the game is finished, whether someone has won it or the game is a draw, the canvas should be cleared and the play board should be drawn again.

Figure 9 shows the canvas draw event handler. It will be triggered when the user starts dragging on the canvas, and that's when we will draw the user input on to the canvas.

A *FirebaseDB.DataChanged* event handler sends two parameters, as follows:

Tag: This is the name of the tag or the key for which data has been updated.

Value: This is the value for the tag.

When this event is triggered, we check to see which piece of data has been changed. If you recall, when drawing on the canvas, we stored *prevX*, *prevY*, *currentX* and *currentY* in CSV format with a key from *KEY_LINE*.

The block in Figure 10 checks to see if the reported tag (key) is *KEY_LINE*. If yes, this means another user is drawing on the canvas. We get the CSV value, convert the CSV into a list using *list from csv row* and store it into our global *csvList* variable. Next, we simply take each piece of data and use it to draw lines on the canvas.

If the tag (key) is *KEY_USER*, it means data has been updated in the cloud. We get the associated value and update our *lblNowDrawing* text to show its related value (the last user's name).

If the tag (key) is *KEY_COMMAND*, we call a stored procedure, the *parseCommand*, to inspect the command. We are using a procedure to reduce the number of blocks in our *DataChanged* event handler.

The procedure *parseCommandBelow* is called from the above blocks. It basically checks to see what command was sent to *FirebaseDB*. If the command was *ERASE*, we use *Canvas.Clear* to clear the canvas. If the command was *COLOR_RED*, we change the paint colour. We do the same for *COLOR_BLUE* and *COLOR_BLACK*.

Now you are done with the block editor too. Next, we will download and install the app on your phone to check how it works.